Lecture 3 (9/18)

Sampling and Simulation with Python

IID samples and simulations

An **IID** sample $X_1, \ldots, X_N$ consists of independent draws from the same distribution.

$$X_i \sim_{\text{iid}} \text{Name}(\text{Parameters})$$

IID samples and simulations

An **IID** sample $X_1, \ldots, X_N$ consists of independent draws from the same distribution.

$$X_i \sim_{\text{iid}} \text{Name}(\text{Parameters})$$

If outcome x occurs $N(x)$ times,

$$P(X = x) \approx \frac{N(x)}{N},$$

and this approximation improves as N grows.

IID samples and simulations

An **IID** sample $X_1, \ldots, X_N$ consists of independent draws from the same distribution.

$$X_i \sim_{\text{iid}} \text{Name}(\text{Parameters})$$

If outcome x occurs $N(x)$ times,

$$P(X = x) \approx \frac{N(x)}{N},$$

and this approximation improves as N grows.

- The error is proportional to $1/\sqrt{N}$, as we discuss more in Unit 3.

IID samples and simulations

An **IID** sample $X_1, \ldots, X_N$ consists of independent draws from the same distribution.

$$X_i \sim_{\text{iid}} \text{Name}(\text{Parameters})$$

If outcome x occurs $N(x)$ times,

$$P(X = x) \approx \frac{N(x)}{N},$$

and this approximation improves as N grows.

- The error is proportional to $1/\sqrt{N}$, as we discuss more in Unit 3.
- A simulation is a means of generating samples from a computer.

IID samples and simulations

An **IID** sample $X_1, \ldots, X_N$ consists of independent draws from the same distribution.

$$X_i \sim_{\text{iid}} \text{Name}(\text{Parameters})$$

If outcome x occurs $N(x)$ times,

$$P(X = x) \approx \frac{N(x)}{N},$$

and this approximation improves as N grows.

- The error is proportional to $1/\sqrt{N}$, as we discuss more in Unit 3.
- A simulation is a means of generating samples from a computer.
- We should **always generate simulated data that has similar properties as real data to test statistical methods**.

Python basics: NumPy arrays

```
import numpy as np  # a "wrapper" for Python lists

x = np.array([3, 1, 4, 1, 5])
y = np.array([1, 9, 4, 1, 5])
z = np.zeros(5) # np.array([0,0,0,0,0])
```

Python basics: NumPy arrays

```
import numpy as np  # a "wrapper" for Python lists

x = np.array([3, 1, 4, 1, 5])
y = np.array([1, 9, 4, 1, 5])
z = np.zeros(5) # np.array([0,0,0,0,0])

x[0]                        # 3
x[1:3]                      # [1, 4]
z[3] = x[1]                 # now z = np.array([0,0,0,1,0])
z[0:2] = np.ones(2)         # now z = np.array([1,1,0,1,0])
```

Python basics: NumPy arrays

```
import numpy as np  # a "wrapper" for Python lists

x = np.array([3, 1, 4, 1, 5])
y = np.array([1, 9, 4, 1, 5])
z = np.zeros(5) # np.array([0,0,0,0,0])

x[0]                        # 3
x[1:3]                      # [1, 4]
z[3] = x[1]                 # now z = np.array([0,0,0,1,0])
z[0:2] = np.ones(2)         # now z = np.array([1,1,0,1,0])

# boolean indexing
x > 2                       # [True,False,True,False,True]
x[x > 2]                    # [3, 4, 5]
x[y <= 1]                   # [3, 1]
x[(x == 1) & (y == 1)]      # [1]
```

Question

```
x = np.array([3, 1, 4, 1, 5])
```

What does each of these evaluate to?

- `x[2]`
- `x[0:2]`
- `x > 3`
- `x[x > 3]`

Question

```
x = np.array([3, 1, 4, 1, 5])
y = np.array([1, 9, 4, 1, 5])
```

Write an expression which returns the sum of the entries of `x` which are greater than or equal to the corresponding entries in `y`.

Question

```
x = np.array([3, 1, 4, 1, 5])
y = np.array([1, 9, 4, 1, 5])
```

Write an expression which returns the sum of the entries of `x` which are greater than or equal to the corresponding entries in `y`.

Question

Suppose `z = np.zeros(5)`. Write one line of code that sets `z[i]` equal to `1` wherever `x[i] == y[i]`.

Python basics: NumPy arrays

```
import numpy as np

x = np.array([3, 1, 4, 1, 5])
y = np.array([1, 9, 4, 1, 5])
```

Python basics: NumPy arrays

```
import numpy as np

x = np.array([3, 1, 4, 1, 5])
y = np.array([1, 9, 4, 1, 5])

len(x)         # 5
np.sum(x)      # 14
np.mean(x)     # 2.8
np.mean(x==1) # 0.4 since True/False are treated as 1/0
```

Python basics: NumPy arrays

```
import numpy as np

x = np.array([3, 1, 4, 1, 5])
y = np.array([1, 9, 4, 1, 5])

len(x)         # 5
np.sum(x)      # 14
np.mean(x)     # 2.8
np.mean(x==1) # 0.4 since True/False are treated as 1/0

# elementwise
x + y          # [4, 10, 8, 2, 10]
x * y          # [3, 9, 16, 1, 25]
```

- `np.mean(x)` equals `np.sum(x) / len(x)`.
- Elementwise operations like `x + y` and `x * y` require arrays of the same length.

Question

```
x = np.array([2, 5, 5, 7, 9])
```

What does each of these print?

- `x[2]`
- `x[x >= 5]`
- `np.mean(x == 5)`
- `len(x[x > 5])`.

Question

```
x = np.array([2, 5, 5, 7, 9])
```

What does each of these print?

- `x[2]`
- `x[x >= 5]`
- `np.mean(x == 5)`
- `len(x[x > 5])`.

Question

Suppose `x` is a numpy array of samples of a random variable. Write Python code to estimate $P(X > 2)$ and $P(X = 4 \mid X > 2)$.

Question

Suppose `x` and `y` are numpy arrays of samples (x_i, y_i) of (X, Y), each taking values in $\{0, 1\}$. How would you compute

- the marginal probability $P(Y = 1)$
- the joint probability $P(X = 1, Y = 0)$
- the conditional probability $P(X = 0 \mid Y = 1)$?

Python basics: Generating samples

```
vals = [0, 1, 2]                    # sample space of X
p = [0.2, 0.3, 0.5]                 # P(X = vals[i])
x = np.random.choice(vals, p=p) # a sample of X
```

Python basics: Generating samples

```
vals = [0, 1, 2]                    # sample space of X
p = [0.2, 0.3, 0.5]                 # P(X = vals[i])
x = np.random.choice(vals, p=p) # a sample of X

# N = 1000 draws from vals
x = np.random.choice(vals, p=p, size=1000)
np.mean(x == 2)          # estimates P(X = 2)
```

Python basics: Generating samples

```
vals = [0, 1, 2]                    # sample space of X
p = [0.2, 0.3, 0.5]                 # P(X = vals[i])
x = np.random.choice(vals, p=p) # a sample of X

# N = 1000 draws from vals
x = np.random.choice(vals, p=p, size=1000)
np.mean(x == 2)          # estimates P(X = 2)

# N draws from a Bernoulli(0.5)
y = np.random.choice([0, 1], p=[0.5, 0.5], size=1000)
np.mean(y)                # estimates P(Y = 1)
```

Practice: simulating a conditional model

Question

Suppose `x` and `y` are generated by:

```
x = np.random.choice([0, 1], p=[0.5, 0.5], size=10000)
p0, p1 = [0.8, 0.2], [0.3, 0.7]   # dist of Y|X=0, Y|X=1
n0, n1 = np.sum(x == 0), np.sum(x == 1)

y = np.zeros(10000, dtype=int)
y[x == 0] = np.random.choice([0, 1], p=p0, size=n0)
y[x == 1] = np.random.choice([0, 1], p=p1, size=n1)
```

Write down the distribution of random variables X and $Y|X$ which are consistent with this code

Practice: simulating a conditional model

Question

Suppose `x` and `y` are generated by:

```
x = np.random.choice([0, 1], p=[0.5, 0.5], size=10000)
p0, p1 = [0.8, 0.2], [0.3, 0.7]   # dist of Y|X=0, Y|X=1
n0, n1 = np.sum(x == 0), np.sum(x == 1)

y = np.zeros(10000, dtype=int)
y[x == 0] = np.random.choice([0, 1], p=p0, size=n0)
y[x == 1] = np.random.choice([0, 1], p=p1, size=n1)
```

Write down the distribution of random variables X and $Y|X$ which are consistent with this code

Question

Let $X \sim \text{Bernoulli}(1/2)$ and $Y \mid X \sim \text{Bernoulli}(0.2 + 0.5X)$. Make Python arrays `x` and `y` which contain samples of these.

Control: If and for

```
# if / else
x = 7
label = 0
if x > 5:
    label = 100
else:
    label = 1
# now label = 100
```

Control: If and for

```
# if / else
x = 7
label = 0
if x > 5:
    label = 100
else:
    label = 1
# now label = 100

# for loop
total = 0
for i in np.array([1, 2, 3, 4]):
    total = total + i
# total = 10
```

Control: If and for

```
# if / else
x = 7
label = 0
if x > 5:
    label = 100
else:
    label = 1
# now label = 100

# for loop
total = 0
for i in np.array([1, 2, 3, 4]):
    total = total + i
# total = 10

# inline for loop (list comprehension)
squares = [2*i for i in range(5)]
# squares = [0, 2, 4, 6, 8]
```

Question

```
x = np.array([2, 5, 1, 8, 3])
total = 0
for v in x[x > 2]:
    total = total + v
```

What is `total` equal to after this loop runs?

Question

```
x = np.array([2, 5, 1, 8, 3])
total = 0
for v in x[x > 2]:
    total = total + v
```

What is `total` equal to after this loop runs?

Question

```
x = np.array([4, -2, 7, -5, 3])
count = 0
for v in x:
    if v < 0:
        count = count + 1
```

What is `count` equal to after this loop runs?

Question

```
x = np.array([2, 5, 1, 8, 3])
total = 0
for v in x[x > 2]:
    total = total + v
```

What is `total` equal to after this loop runs?

Question

```
x = np.array([4, -2, 7, -5, 3])
count = 0
for v in x:
    if v < 0:
        count = count + 1
```

What is `count` equal to after this loop runs?

Question

Write a list comprehension that produces `[1, 4, 9, 16, 25]` using `range(1, 6)`.

Python basics: functions

`def` defines a reusable block of code that takes inputs (*arguments*) and can `return` an output.

```
def estimate_prob(x, value):
    return np.mean(x == value)

x = np.array([0, 1, 1, 0, 1])
estimate_prob(x, 1)   # 0.6
estimate_prob(x, 0)   # 0.4
```

- `x` and `value` are just local names for whatever gets passed in when the function is called.

What you need to know for midterm 1

For exams you should know all the things we've covered so far:

- Working with 1 dimensional numpy arrays (conditional indexing, finding length, mean)
- Generating random samples with `np.random` and **translating code to probability models**.
- Control structures: For loops, if statements and functions (only simple function definitions without keyword args, no fancy stuff!)

What you need to know for midterm 1

For exams you should know all the things we've covered so far:

- Working with 1 dimensional numpy arrays (conditional indexing, finding length, mean)
- Generating random samples with `np.random` and **translating code to probability models**.
- Control structures: For loops, if statements and functions (only simple function definitions without keyword args, no fancy stuff!)

For assignments, I expect you can figure out more complex operations with a combination of online resources and LLMs:

- Multidimensional arrays
- Plotting
- Dataframes

Example

$P(C, M)$	$M = 1$	$M = 0$
$C = 1$	0.0021	0.0019
$C = 0$	0.4899	0.5061

Example

$P(C, M)$	$M = 1$	$M = 0$
$C = 1$	0.0021	0.0019
$C = 0$	0.4899	0.5061

- Previously we saw that the conditional probabilities differ from the marginals, suggesting they are not independent

Example

$P(C, M)$	$M = 1$	$M = 0$
$C = 1$	0.0021	0.0019
$C = 0$	0.4899	0.5061

- Previously we saw that the conditional probabilities differ from the marginals, suggesting they are not independent
- But are we likely to see this by chance in a small sample? This is what Fisher's exact test is for (you don't need to know that).

Example

$P(C, M)$	$M = 1$	$M = 0$
$C = 1$	0.0021	0.0019
$C = 0$	0.4899	0.5061

- Previously we saw that the conditional probabilities differ from the marginals, suggesting they are not independent
- But are we likely to see this by chance in a small sample? This is what Fisher's exact test is for (you don't need to know that).
- We can test with simulations instead: Plan
  - Write a function `sampleindependent` which generates data under the null hypothesis that they are independent
  - See how likely it is for the `samplejoint` to produce the probabilities in the table
  - We will also writ a function `samplejoint` which generates data from the probabilities in the table, just to compare the code.

Codes $1, 2, 3, 4$ stand for $(C, M) = (0, 0), (0, 1), (1, 0), (1, 1)$, in the same order as the table.

```
codes = [1, 2, 3, 4]                        # (C,M) pair index
probs = [0.5061, 0.4899, 0.0019, 0.0021]  # matches the table

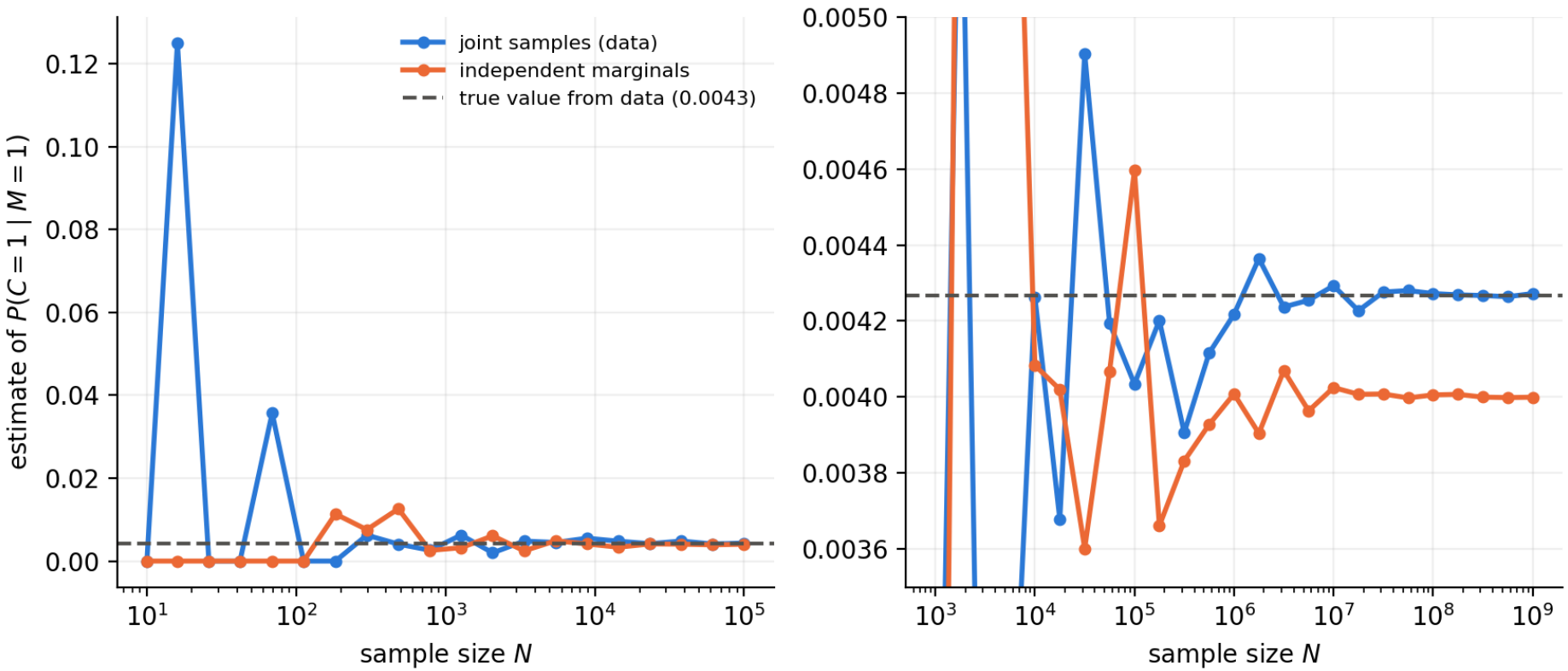
def samplejoint(N):
    oc = np.random.choice(codes, p=probs, size=N)
    C = np.zeros(N, dtype=int)
    M = np.zeros(N, dtype=int)
    C[oc == 3] = 1   # codes 3 and 4 both have C = 1
    C[oc == 4] = 1
    M[oc == 2] = 1   # codes 2 and 4 both have M = 1
    M[oc == 4] = 1
    return C, M

p_C, p_M = 0.004, 0.492   # marginal probabilities from the table

def sampleindependent(N):
    C = np.random.choice([0, 1], p=[1 - p_C, p_C], size=N)
    M = np.random.choice([0, 1], p=[1 - p_M, p_M], size=N)
    return C, M
```

Checking independence by simulation

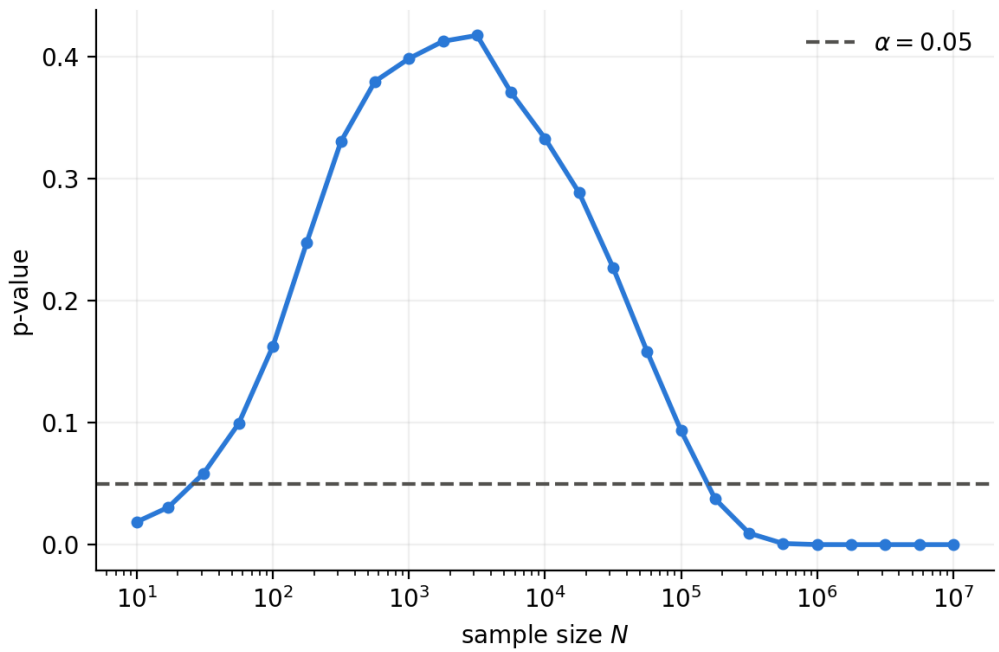
The plots below compare $P(C = 1 \mid M = 1)$ estimated from `samplejoint` versus `sampleindependent`, as the sample size N grows: the two are indistinguishable until N is large.



The true difference between the conditional and marginal probabilities in the table is

$$P(C = 1 \mid M = 1) - P(C = 1) = \frac{0.0021}{0.492} - 0.004 \approx 0.00027.$$

For each sample size N, we draw many replicate datasets from `sampleindependent(N)` and compute this same difference in each one.



Question

Recall the Markov chain from Lecture 2:

$$X_0 \sim \text{Bernoulli}(1/2), \qquad X_{t+1} \mid X_t \sim \text{Bernoulli}(q_1 X_t + q_0(1 - X_t)).$$

Write code to simulate $X_0, \ldots, X_{T-1}$, then estimate $\lim_{t\to\infty} P(X_t = 1)$ once the chain has settled down.

Question

Recall the Markov chain from Lecture 2:

$$X_0 \sim \text{Bernoulli}(1/2), \qquad X_{t+1} \mid X_t \sim \text{Bernoulli}(q_1 X_t + q_0(1 - X_t)).$$

Write code to simulate $X_0, \ldots, X_{T-1}$, then estimate $\lim_{t\to\infty} P(X_t = 1)$ once the chain has settled down.

```
q0, q1 = 0.2, 0.7
T = 1000
x = np.zeros(T, dtype=int)
x[0] = np.random.choice([0, 1], p=[0.5, 0.5])
for t in range(1, T):
    if x[t - 1] == 1:
        x[t] = np.random.choice([0, 1], p=[1 - q1, q1])
    else:
        x[t] = np.random.choice([0, 1], p=[1 - q0, q0])

np.mean(x[500:])   # should be close to q0 / (1 - q1 + q0)
```

Solutions: NumPy indexing

For `x = np.array([3, 1, 4, 1, 5])`:

- `x[2]` → `4`
- `x[0:2]` → `[3, 1]`
- `x > 3` → `[False, False, True, False, True]`
- `x[x > 3]` → `[4, 5]`

For `x = np.array([3, 1, 4, 1, 5])`, `y = np.array([1, 9, 4, 1, 5])`:

- Sum of entries of `x` at least the corresponding entry of `y`: `np.sum(x[x >= y])` → `13`
- Setting `z[i] = 1` wherever `x[i] == y[i]`: `z[x == y] = 1` → `z = [0, 0, 1, 1, 1]`

Solutions: NumPy arrays and probability

For `x = np.array([2, 5, 5, 7, 9])`:

- `x[2]` $\rightarrow$ `5`
- `x[x >= 5]` $\rightarrow$ `[5, 5, 7, 9]`
- `np.mean(x == 5)` $\rightarrow$ `0.4`
- `len(x[x > 5])` $\rightarrow$ `2`

Estimating probabilities from a long sample `x`:

- $P(X > 2)$: `np.mean(x > 2)`
- $P(X = 4 \mid X > 2)$: `np.mean(x[x > 2] == 4)`

For paired samples `x`, `y` of $(X, Y) \in \{0, 1\}^2$:

- $P(Y = 1)$: `np.mean(y == 1)`
- $P(X = 1, Y = 0)$: `np.mean((x == 1) & (y == 0))`
- $P(X = 0 \mid Y = 1)$: `np.mean(x[y == 1] == 0)`

Solutions: simulating a conditional model

The given code has $X \sim \text{Bernoulli}(1/2)$, and since `p0 = [0.8, 0.2]` gives $P(Y{=}1 \mid X{=}0) = 0.2$ and `p1 = [0.3, 0.7]` gives $P(Y{=}1 \mid X{=}1) = 0.7$, it is consistent with

$$X \sim \text{Bernoulli}(1/2), \qquad Y \mid X \sim \text{Bernoulli}(0.2 + 0.5X).$$

This is exactly the model in the next example, so the same code answers both questions:

```
x = np.random.choice([0, 1], p=[0.5, 0.5], size=N)
p0, p1 = [0.8, 0.2], [0.3, 0.7]   # dist of Y|X=0, Y|X=1
n0, n1 = np.sum(x == 0), np.sum(x == 1)

y = np.zeros(N, dtype=int)
y[x == 0] = np.random.choice([0, 1], p=p0, size=n0)
y[x == 1] = np.random.choice([0, 1], p=p1, size=n1)
```

Solutions: control flow

- `total` → 16 (the loop sums `x[x > 2] = [5, 8, 3]`)
- `count` → 2 (two negative entries: `-2` and `-5`)
- `[i**2 for i in range(1, 6)]` → `[1, 4, 9, 16, 25]`

For the Markov chain simulation, see the code on the earlier slide; `np.mean(x[500:])` should be close to $q_0/(1 - q_1 + q_0) = 0.2/0.5 = 0.4$.